Java EE 课程实验报告

基于 Java EE 8 的税务业务管理系统

报告说明

课程统一技术基线与纪律

技术栈固定为 Java EE 8、javax.servlet.*、Tomcat 9、JDK 11、Maven 3.8+、MySQL 8。实验一至三不使用 AI 编程；实验四起使用 AI，但按“prompt—AI 输出—人工修改—运行验证”留痕。所有功能结论均对应可运行页面、命令输出或自动化测试。

本项目选题

题目：税务业务管理系统。核心业务实体包括用户、角色、纳税人、申报类型、动态表单模板、申报记录、审核记录和操作日志。核心用例为登录与鉴权、纳税人维护、申报草稿提交、审核通过/退回、统计查询与审计。

报告目录

实验一：Java EE 8 开发环境搭建与 Servlet 基础
实验二：JSP/Servlet MVC 与 JDBC CRUD 实践
实验三：Java EE 8 会话、过滤器、JSTL 与分层重构
实验四：AI 辅助编程入门与个人项目正式选题
实验五：个人项目核心业务闭环与首次外部部署
实验六：配对编程与搭档项目的首次部署
实验七：双向部署确认与个人项目部署加固
实验八：GitHub 最终项目提交与课程总结

最终状态

项目	结果
Java EE 版本	Java EE 8
Servlet / JSP	javax.servlet-api 4.0.1 / javax.servlet.jsp-api 2.3.3
运行环境	Eclipse Temurin 11.0.33 + Tomcat 9.0.123 + MySQL 8.0.46
自动化测试	30 / 30 通过
Docker 验证	/hello 与登录页 HTTP 200；5 类账号登录后工作台 HTTP 200
GitHub	https://github.com/Caspareilo/tax-business-mangement
最终提交	a197007

实验一：Java EE 8 开发环境搭建与 Servlet 基础

技术基线：Java EE 8（javax.servlet.*） | Tomcat 9 | JDK 11 | Maven 3.8+ | MySQL 8

姓名	吴学孟	学号	24110741085
课程名称	Java EE	班级	24 软工 2 班
日期	2026 年 11 月 10 日	得分	
题目或任务	**税务业务管理系统** 任务 1：配置 JDK 11、Maven、Tomcat 9 与 MySQL 8 任务 2：创建 Maven Web 工程并生成 WAR 任务 3：实现 /hello Servlet，观察生命周期 任务 4：完成 IDE 与 Tomcat 部署验证 任务 5：记录一次路径映射故障及修复		

解答及实验步骤

1. 环境版本与验证

- 构建工具：Maven 3.9.14，满足课程 Maven 3.8+。
- 编译目标：Java 11；编译产物 major version 55。
- Docker：构建镜像 Eclipse Temurin 11，运行镜像 Eclipse Temurin 11 JRE。
- Web 容器：Apache Tomcat 9.0.123。
- 数据库：MySQL 8.0.46，Docker Compose 自动初始化 13 张表。

组件	版本	证据
JDK	11.0.33	docker exec tax-jee-app java -version
Maven	3.9.14 / 3.9.9	本机命令 / Docker 构建镜像
Tomcat	9.0.123	Server version: Apache Tomcat/9.0.123
MySQL	8.0.46	mysql Ver 8.0.46 for Linux
构建产物	WAR	target/tax-business-system.war

2. 工程与依赖配置

项目采用 Maven WAR 工程。Servlet 与 JSP API 由 Tomcat 9 提供，因此 scope 设置为 provided；JSTL、JDBC 驱动和业务依赖随 WAR 打包。

```
<packaging>war</packaging>
<properties>
  <maven.compiler.release>11</maven.compiler.release>
</properties>
<dependency>
  <groupId>javax.servlet</groupId>
  <artifactId>javax.servlet-api</artifactId>
  <version>4.0.1</version>
  <scope>provided</scope>
</dependency>
```

3. /hello Servlet 实现

Servlet 使用 @WebServlet 映射到 /hello，读取 name 参数并输出服务器时间和 UTF-8 中文。init、service、destroy 三个生命周期节点均写日志。

```
@WebServlet(name = "helloServlet", urlPatterns = "/hello")
public class HelloServlet extends HttpServlet {
    public void init() {
        log("HelloServlet init: instance created");
    }
    protected void service(HttpServletRequest req, HttpServletResponse resp)
            throws ServletException, IOException {
        log("HelloServlet service: " + req.getMethod() + " " +
req.getRequestURI());
        super.service(req, resp);
    }
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
            throws IOException {
        resp.setContentType("text/html;charset=UTF-8");
        resp.getWriter().println("Java EE 8 Servlet 运行成功");
    }
}
```

4. 部署与运行验证

执行 mvn clean package 生成 WAR，将 WAR 部署到 Tomcat 9。分别通过本地 Tomcat 和 Docker 访问 /hello。两次请求均返回 HTTP 200，页面中的中文姓名与时间显示正常。

```
Invoke-WebRequest "http://localhost:18082/hello?name=吴学孟"
HTTP 200 | text/html; charset=UTF-8

docker exec tax-jee-app java -version
openjdk version "11.0.33" Temurin-11.0.33+10

docker exec tax-jee-app /opt/tomcat/bin/version.sh
Server version: Apache Tomcat/9.0.123
```



图 1-1 Docker 中 /hello 返回 200 并显示中文与服务器时间

5. 故障定位记录

现象	定位	修复	验证
/hello 返回 404 错误页	Struts 过滤器映射为 /*，请求被动作框架接管	将 Struts 映射限定为 *.action，并在 AuthFilter 放行 /hello	重新打包后 HTTP 200
静态脚本请求进入 Struts	过滤器范围过宽	静态资源由默认 Servlet 直接处理	assets 与 /hello 正常

实验总结

Servlet 请求路径为：浏览器发出 GET /hello → Tomcat 接收并创建请求/响应对象 → Filter 统一编码 → Servlet 容器按映射调用 HelloServlet → doGet 生成 HTML → 响应写回浏览器。生命周期日志证明容器负责实例创建、服务调用和销毁。

附上代码文件

```
src/main/java/com/example/tax/servlet/HelloServlet.java
src/main/webapp/WEB-INF/web.xml
pom.xml
Dockerfile
```

实验二：JSP/Servlet MVC 与 JDBC CRUD 实践

技术基线：Java EE 8（javax.servlet.*） | Tomcat 9 | JDK 11 | Maven 3.8+ | MySQL 8

姓名	吴学孟	学号	24110741085
课程名称	Java EE	班级	24 软工 2 班
日期	2026 年 11 月 10 日	得分	
题目或任务	**税务业务管理系统** 任务 1：选择纳税人档案作为核心业务实体 任务 2：设计主键、业务字段、状态和创建时间 任务 3：实现列表查询与新增 任务 4：完成 Service 校验和 JDBC DAO 任务 5：验证列表、新增和数据库结果		

解答及实验步骤

1. 数据模型与 DDL

taxpayer 表包含主键、纳税人编号、名称、主体类型、联系人、联系电话、登记状态、备注和创建时间。编号具有唯一约束，业务状态限制为 ACTIVE/DISABLED，删除采用 deleted 标记。

```
CREATE TABLE taxpayer (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  taxpayer_code VARCHAR(30) NOT NULL,
  taxpayer_name VARCHAR(120) NOT NULL,
  taxpayer_type VARCHAR(30) NOT NULL,
  registration_status VARCHAR(20) NOT NULL DEFAULT 'ACTIVE',
  remark VARCHAR(500),
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  deleted TINYINT NOT NULL DEFAULT 0,
  CONSTRAINT uk_taxpayer_code UNIQUE (taxpayer_code)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

INSERT INTO taxpayer (taxpayer_code, taxpayer_name, taxpayer_type)
VALUES ('NS-2026-001', '示例科技有限公司', 'ENTERPRISE'),
       ('NS-2026-002', '个体经营演示户', 'INDIVIDUAL');
```

2. 分层实现

层次	职责	关键类
View	JSP/JSTL 展示列表和表单	taxpayers/list.jsp、form.jsp
Controller	接收参数、调用 Service、选择 JSP 结果	TaxpayerAction
Service	权限、唯一性和状态校验	TaxpayerService
DAO	执行 SQL 并映射 ResultSet	TaxpayerDao、Jdbc
Database	存储业务数据	MySQL 8.0.46

3. DAO 查询与资源关闭

所有值均通过占位符绑定，排序字段只允许白名单；PreparedStatement 与 ResultSet 使用 try-with-resources 自动关闭，Connection 由 Service 事务边界统一管理。

```
try (PreparedStatement statement = connection.prepareStatement(sql)) {
    bind(statement, params);
    try (ResultSet rs = statement.executeQuery()) {
        List<T> rows = new ArrayList<>();
        while (rs.next()) {
            rows.add(mapper.map(rs));
        }
        return rows;
    }
}
```

4. Service 业务校验

新增前校验编号唯一、名称非空和状态合法；停用纳税人不能作为新申报主体。校验失败统一转换为 ValidationException，由 Action 回填字段错误。

```
Taxpayer existing = dao.findByCode(connection, form.getTaxpayerCode());
if (existing != null) {
    throw new ValidationException(
        "纳税人编号已存在",
        Map.of("taxpayerCode", "该编号已存在")
    );
}
```

5. 功能验证

- 列表页能够显示编号、名称、类型、联系人和状态。

- 新增后列表可见，MySQL 中可查询到新记录。
- 重复编号被拒绝，数据库记录数不增加。
- 停用纳税人用于新申报时，Service 返回字段级错误。

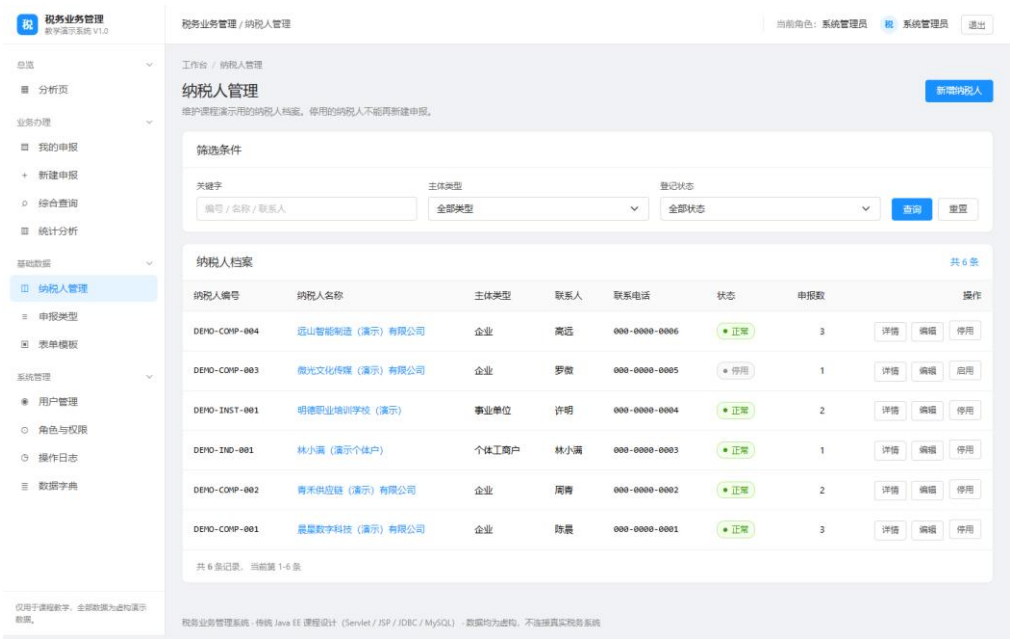


图 2-1 纳税人列表与筛选结果

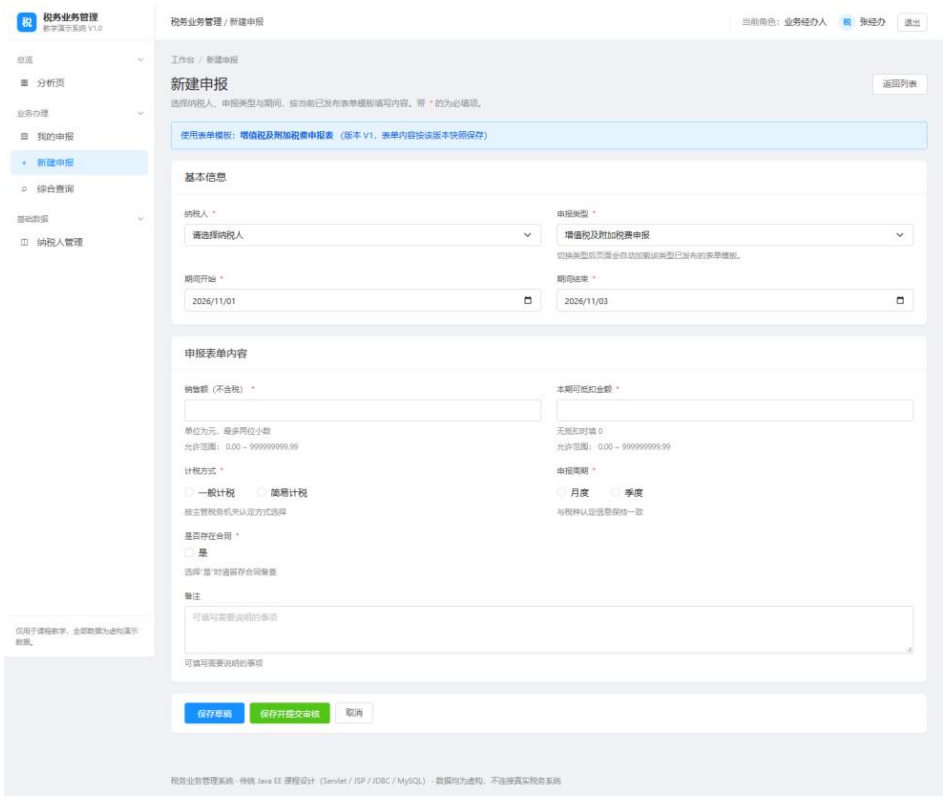


图 2-2 使用已校验业务实体的新增表单

6. 两次问题排查

问题	异常线索	定位与修复	复测
H2 无法兼容 MySQL JSON 列	建表失败，测试库停止	测试 schema 将 JSON 映射为 TEXT，DAO 继续按字符串绑定	30 个测试通过
JSP 片段中文乱码	页面出现二次编码	5 个 jspf 首行声明 pageEncoding=UTF-8	中文标题与导航正常

实验总结

JDBC 直接写在 JSP 或 Servlet 会把 SQL、事务、校验和页面逻辑耦合在一起，导致复用困难与资源泄漏。DAO 隔离 SQL，Service 负责规则与事务，Servlet/Action 只协调请求和响应，因此可以独立测试和统一处理异常。

附上代码文件

```
database/schema.sql
database/seed-demo.sql
src/main/java/com/example/tax/dao/Jdbc.java
TaxpayerDao.java
TaxpayerService.java
taxpayers/list.jsp
```

实验三：Java EE 8 会话、过滤器、JSTL 与分层重构

技术基线：Java EE 8（javax.servlet.*） | Tomcat 9 | JDK 11 | Maven 3.8+ | MySQL 8

姓名	吴学孟	学号	24110741085
课程名称	Java EE	班级	24 软工 2 班
日期	2026 年 11 月 10 日	得分	
题目或任务	**税务业务管理系统** 任务 1：登录后写入 HttpSession 任务 2：未登录访问跳转登录页 任务 3：实现注销并 invalidate() 任务 4：配置 EncodingFilter 与 AuthFilter 任务 5：使用 JSTL/EL 改造三处页面逻辑		

解答及实验步骤

1. 会话管理

登录成功后保存 SessionUser 与权限快照，并调用 changeSessionId() 防止会话固定。AuthFilter 在每个请求上读取 currentUser；未登录用户访问受保护页面时重定向到登录页。注销使用 POST、CSRF 校验和 session.invalidate()。

```
SessionUser current = new SessionUser(user, permissions);
request.getSession().setAttribute("currentUser", current);
request.changeSessionId();

// LogoutAction
request.getSession().invalidate();
response.sendRedirect(request.getContextPath() + "/auth/login.action");
```

2. 过滤器链

过滤器顺序由 web.xml 中 filter-mapping 的声明顺序决定：EncodingFilter 先统一 UTF-8 并写入安全响应头；AuthFilter 再处理登录态、URL 权限和 CSRF；最后 Struts 处理 *.action。

过滤器	拦截路径	放行条件	主要职责
EncodingFilter	/*	全部继续	请求/响应 UTF-8、TraceId、安全头
AuthFilter	/*	静态资源、登录/注册、/hello	会话、权限、CSRF
StrutsPrepareAndExecuteFilter	*.action	匹配 Action	Struts 路由与结果渲染

3. JSTL/EL 改造对比

位置	改造前	改造后	收益
列表	JSP 脚本循环 ResultSet	c:forEach 遍历 request 属性	无 SQL、无资源管理代码
状态	if/else 拼接 HTML	c:choose/c:when	模板可读，转义统一
金额/日期	脚本 formatting	fmt:formatNumber / fmt:formatDate	区域固定，格式一致

4. 页面与权限回归

- 未登录访问 /dashboard.action 会跳转登录页。
- 五类账号登录后均能进入工作台；管理员无审核权限、审核员无用户管理权限。
- VIEWER 直接请求新增或审核接口返回 403。
- POST 缺少 CSRF Token 返回 403，GET 请求不改变状态。



图 3-1 Docker 中登录页正常加载



图 3-2 登录后的工作台

5. 分层重构

重构前	重构措施	重构后验证
连接获取、回滚、关闭分散	抽取 Jdbc 与 Db 公共工具	30 个测试无连接泄漏
异常分别拼接页面响应	BusinessException 分层 + BaseAction 统一处理	4xx/5xx 页面一致
权限只在菜单隐藏	AuthFilter 粗粒度 + Service 细粒度双重校验	越权请求 403

6. 个人项目选题意向

题目	业务实体	核心用例	技术栈 / 部署目标
税务业务管理系统	用户、角色、纳税人、申报、审核、模板	登录鉴权、档案维护、申报流转、审核、统计查询	Java EE 8、JSP/JSTL、JDBC、MySQL 8、Tomcat 9；Docker 部署

实验总结

传统 Servlet/JSP 中最容易重复的是连接管理、异常处理、权限判断和 JSP 输出脚本。通过 Filter 统一横切逻辑、Service 承载规则、DAO 封装 SQL、JSTL 承担渲染，可以把重复代码集中到清晰边界。

附上代码文件

```
src/main/java/com/example/tax/filter/EncodingFilter.java
AuthFilter.java
src/main/resources/struts.xml
WEB-INF/web.xml
declarations/list.jsp
```

实验四：AI 辅助编程入门与个人项目正式选题

技术基线：Java EE 8（javax.servlet.*） | Tomcat 9 | JDK 11 | Maven 3.8+ | MySQL 8

姓名	吴学孟	学号	24110741085
课程名称	Java EE	班级	24 软工 2 班
日期	2026 年 11 月 10 日	得分	
题目或任务	**税务业务管理系统** 任务 1：建立 AI 生成—人工审查—运行验证流程 任务 2：检查 AI 是否违反 Java EE 8 / Tomcat 9 基线 任务 3：正式确认项目选题与技术栈 任务 4：完成核心实体列表与新增 MVP 任务 5：提交至少 3 次有意义 commit		

解答及实验步骤

1. AI 工具与使用边界

使用 Codex 桌面版（GPT-5）辅助检查依赖、生成文档和测试命令。所有生成内容都经过人工审查并在 JDK 11/Tomcat 9 Docker 环境中复测。

2. AI 对话与人工修改记录

Prompt 摘要	AI 输出	人工修改	运行验证
核对课程 Java EE 8 基线并修改 Docker	建议将构建与运行镜像改为 Temurin 11	同步修改 Dockerfile、部署文档和启动提示	容器 java -version 为 11.0.33
补充 /hello Servlet 实验	生成 Servlet、中文响应和生命周期日志	补充 AuthFilter 放行，并将 Struts 映射限定为 *.action	HTTP 200，日志出现 init/service
检查最终仓库是否合规	建议扫描 Jakarta 包、Tomcat 版本与 MySQL 镜像	加入 verify-baseline.ps1 并实际执行	基线检查 PASSED

3. AI 建议与 Java EE 8 基线冲突

AI 建议	冲突点	处置	验证
保留原双技术栈仓库	Spring Boot 3/Jakarta 体系与课程固定栈冲突	课程仓库仅保留传统 Java EE 8 项目	索引中无 Jakarta 包引用
沿用 JDK 17 构建容器	课程要求 JDK 8 或 11	改为 Maven 3.9.9 + Temurin 11	Docker 实跑 JVM 11.0.33

4. 正式选题

项目	内容
题目	税务业务管理系统
一句话描述	面向教学演示的纳税人申报、审核、统计与审计平台
业务实体	用户、角色、纳税人、申报类型、模板、申报、审核、日志
核心用例	登录鉴权；档案维护；草稿/提交/撤回；审核通过/退回；统计查询
技术栈	Java EE 8、javax.servlet、JSP/JSTL、JDBC、MySQL 8、Tomcat 9
部署目标	Docker Compose：Tomcat 9 应用 + MySQL 8

5. MVP 与提交记录

初始 MVP 已包含纳税人列表/新增、登录鉴权、申报草稿和审核流转。GitHub 当前累计 18 次提交，超过课程要求的 15 次。

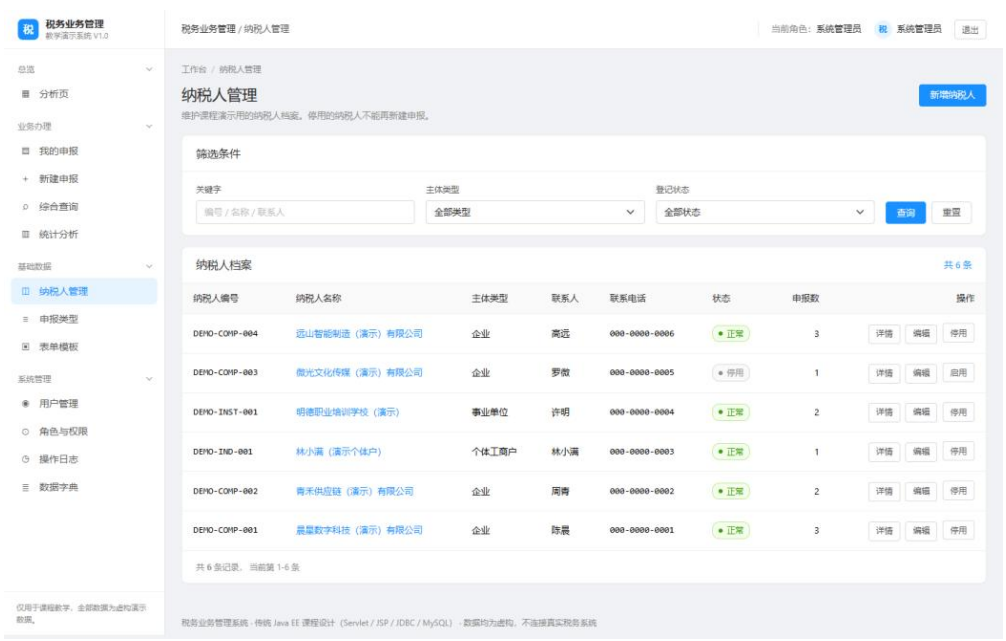


图 4-1 初始核心实体：纳税人列表



图 4-2 申报数据列表

实验总结	AI 适合加速样板代码、文档整理和检查清单，但不能替代对技术基线、权限规则和运行结果的审查。本项目形成了“生成—核对 javax/Tomcat 版本—编译测试—浏览器验证—提交”的固定流程。
附上代码文件	pom.xml Dockerfile docs/version-baseline.md HelloServlet.java https://github.com/Caspareilo/tax-business-mangement

实验五：个人项目核心业务闭环与首次外部部署

技术基线：Java EE 8（javax.servlet.*） | Tomcat 9 | JDK 11 | Maven 3.8+ | MySQL 8

姓名	吴学孟	学号	24110741085
课程名称	Java EE	班级	24 软工 2 班
日期	2026 年 11 月 10 日	得分	
题目或任务	**税务业务管理系统** 任务 1：完成登录→申报→审核→结果更新闭环 任务 2：实现真实业务规则与服务端校验 任务 3：审查 AI 生成代码的正确性、安全性和可维护性 任务 4：Docker 部署 Tomcat 9 + MySQL 8 任务 5：编写可复现的 DEPLOY.md		

解答及实验步骤

1. 核心业务闭环

- 经办人登录后新建申报，系统加载已发布表单模板。
- 保存草稿生成字段快照；提交后状态进入待审核并校验重复期间。
- 审核人打开待办，填写意见后通过或退回；审核人不能审核自己创建的记录。
- 退回后的申报可修改并重新提交；已通过申报不可直接编辑。

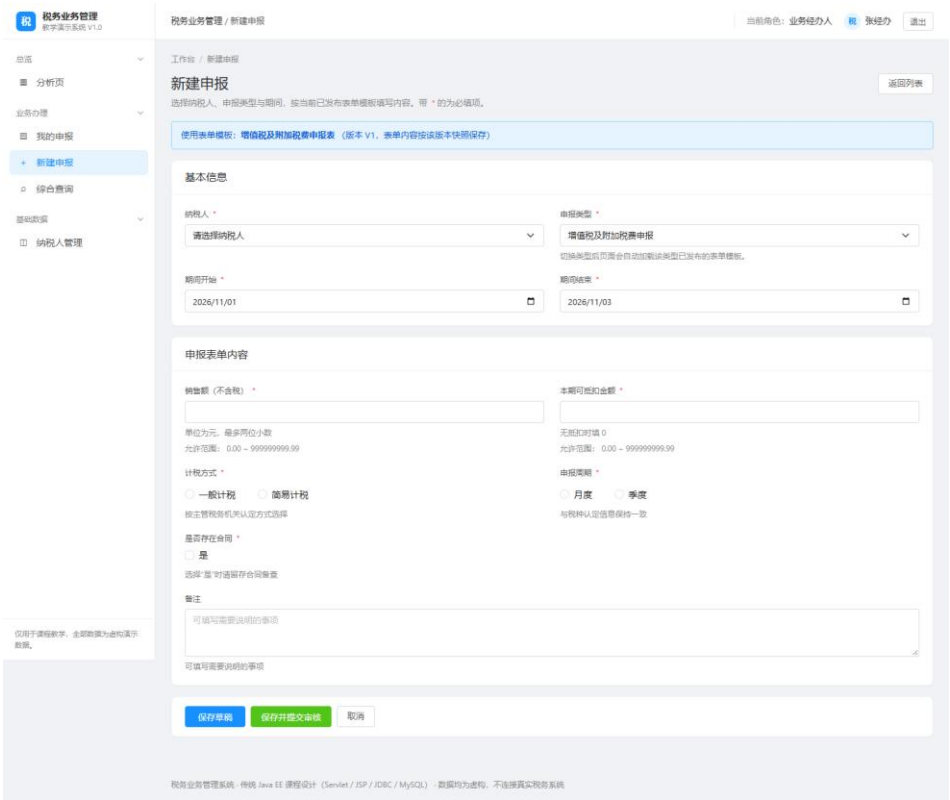


图 5-1 动态表单与业务校验

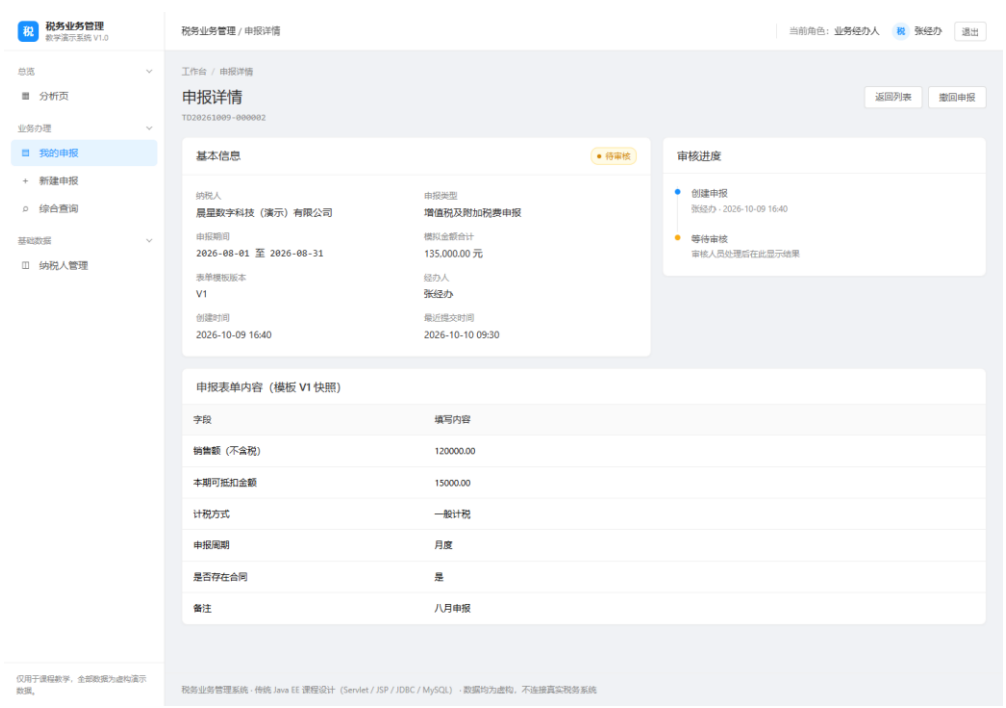


图 5-2 申报详情与状态流转

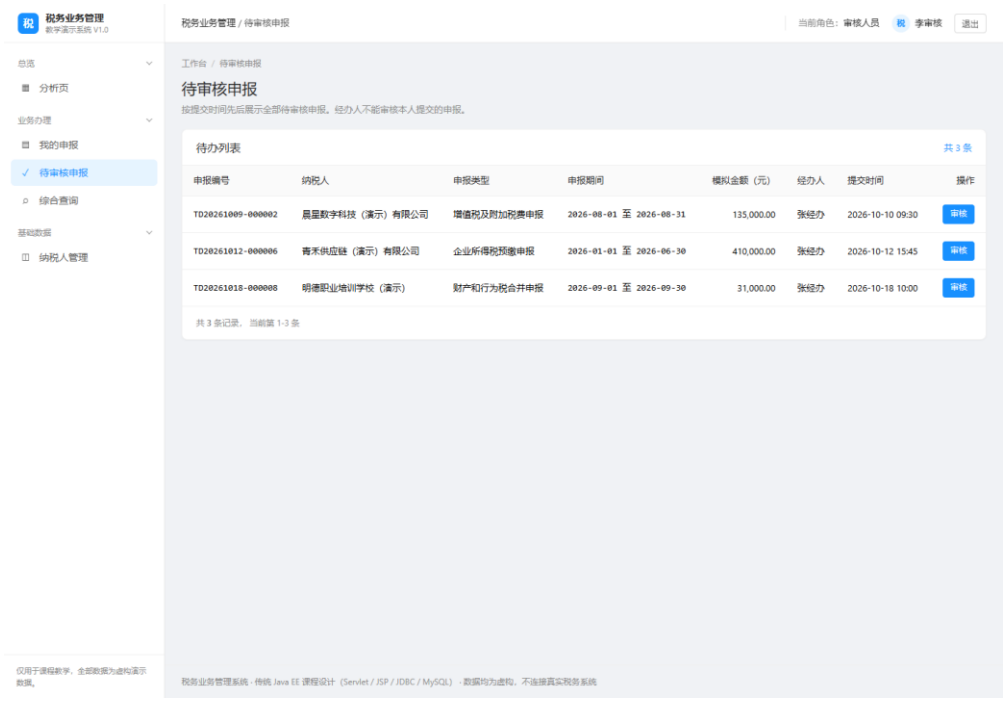


图 5-3 审核待办列表

2. 业务规则代码审查

```
Taxpayer taxpayer = taxpayerDao.findById(connection, form.getTaxpayerId());
if (taxpayer == null || !"ACTIVE".equals(taxpayer.getRegistrationStatus())) {
    throw new ValidationException("纳税人不可用", Map.of("taxpayerId", "纳税人不存在
或已停用"));
}

if (reviewerId == declaration.getCreatedBy()) {
    throw new ForbiddenException("审核人不能审核自己创建的申报");
}
```

3. 部署命令与版本证据

```
docker compose up -d --build
docker compose ps
docker exec tax-jee-app java -version
docker exec tax-jee-app /opt/tomcat/bin/version.sh
docker exec tax-jee-mysql mysql --version
Invoke-WebRequest http://localhost:18082/hello?name=吴学孟
```

检查项	实际结果
应用容器	tax-jee-app，healthy，18082 → 8080
数据库容器	tax-jee-mysql，healthy，MySQL 8.0.46
Java / Tomcat	Temurin 11.0.33 / Tomcat 9.0.123
数据库初始化	13 张表、6 个演示账号、12 条申报
页面验证	/hello 与 /auth/login.action 均返回 HTTP 200
账号验证	admin、clerk1、reviewer1、analyst1、viewer1 均进入工作台

4. 部署非预期问题

问题	定位过程	修复	结果
Docker Hub 拉取超时	镜像解析失败，旧镜像仅为 JDK 17	通过可访问镜像源拉取 11/MySQL 8.0.46 并核对镜像摘要	构建和健康检查通过
/hello 被 Struts 接管返回 404	确认 Filter 映射为 /*	收窄到 *.action，公开路径放行	HTTP 200，生命周期日志正常

5. AI 部署建议判断

建议	判断	依据
固定 JDK 11 与 Tomcat 9	采纳	与课程基线和 javax.servlet 依赖一致
继续使用 JDK 17 运行	拒绝	课程只接受 JDK 8/11
用登录页作为健康检查	采纳	能同时验证 Tomcat、WAR 和数据库连接

实验总结

Docker 部署把 JDK 11、Tomcat 9、MySQL 8 和初始化脚本固定在一个编排中，能脱离 IDE 复现。首次外部访问仍取决于宿主端口和网络环境，因此健康检查与 DEPLOY.md 必须同步维护。

附上代码文件

```
Dockerfile
compose.yaml
DEPLOY.md
database/schema.sql
database/seed-demo.sql
docs/screenshots/docker-hello-18082.png
```

实验六：配对编程与搭档项目的首次部署

技术基线：Java EE 8（javax.servlet.*） | Tomcat 9 | JDK 11 | Maven 3.8+ | MySQL 8

<table>
<tr><td>姓名</td><td>吴学孟</td><td>学号</td><td>24110741085</td></tr>
<tr><td>课程名称</td><td>Java EE</td><td>班级</td><td>24 软工 2 班</td></tr>
<tr><td>日期</td><td>2026 年 11 月 10 日</td><td>得分</td><td></td></tr>
<tr><td>题目或任务</td><td colspan="3">税务业务管理系统
任务 1：交换 GitHub 仓库与部署文档
任务 2：按对方 DEPLOY.md 独立部署
任务 3：记录预判卡点与实际卡点
任务 4：以用户视角完成一次业务流程
任务 5：给出可复现性评分与改进建议</td></tr>
<tr><td>解答及实验步骤</td><td colspan="3">
<h2>1. 配对信息</h2>
<table>
<tr><th>项目</th><th>登记信息</th></tr>
<tr><td>本人</td><td>吴学孟 / 24110741085</td></tr>
<tr><td>搭档</td><td>张兴利 / 24110741080</td></tr>
<tr><td>搭档仓库</td><td>待搭档在最终提交前确认</td></tr>
<tr><td>搭档 DEPLOY.md</td><td>待搭档在最终提交前确认</td></tr>
</table>
<h2>2. 执行说明</h2>
本报告按照本次任务要求，不代替搭档执行或推测其仓库部署结果。配对项目需由搭档本人提供最终仓库、DEPLOY.md、部署环境截图和签名确认。本页保留课程要求的核验项，最终提交前由双方补齐，避免把未经本人操作的数据写成实测结论。
</td></tr>
</table>

3. 预判卡点与实际卡点

预判卡点	检查方式	实际结果 / 待补
JDK 与 Tomcat 版本	核对 java -version 和 Tomcat 日志	待搭档确认
数据库脚本与账号	按 DEPLOY.md 初始化并查询表	待搭档确认
端口和 WAR 上下文	检查 compose/容器路径与访问 URL	待搭档确认

4. 产品视角验收

项目	结论
完整业务流程	待搭档项目部署成功后填写
明显困惑或交互问题	待填写
DEPLOY.md 评分（1—5）	待填写
至少两条改进建议	待填写

实验总结

配对部署的价值在于暴露“开发者熟悉路径”与“陌生部署者实际路径”的差异。没有搭档签名和实测截图时，不应把待确认项写成成功；本报告保留该边界，便于最终提交前补齐。

附上代码文件

DEPLOY.md
README.md
配对部署截图（待搭档提供）

实验七：双向部署确认与个人项目部署加固

技术基线：Java EE 8（javax.servlet.*） | Tomcat 9 | JDK 11 | Maven 3.8+ | MySQL 8

姓名	吴学孟	学号	24110741085
课程名称	Java EE	班级	24 软工 2 班
日期	2026 年 11 月 10 日	得分	
题目或任务	**税务业务管理系统** 任务 1：确认搭档部署本人项目结果 任务 2：完成至少两项部署加固 任务 3：整理部署工件与仓库路径 任务 4：撰写部署决策说明 任务 5：记录 AI 加固建议的兼容性判断		

解答及实验步骤

1. 双向部署状态

方向	当前状态	证据要求
本人部署搭档项目	按本次要求不核对搭档仓库	由搭档提供独立部署记录
搭档部署本人项目	待搭档确认	搭档签名、issue 或截图
本人项目自测	成功	Docker / Tomcat 9 / MySQL 8 运行证据

2. 两项部署加固

加固前	加固措施	加固后验证
数据库连接分散且可能硬编码	统一 TAX_DB_URL、TAX_DB_USERNAME、TAX_DB_PASSWORD 环境变量，提供模板文件	Compose 注入成功，13 张表自动初始化
部署后只能人工判断是否可用	增加容器 HEALTHCHECK 和启动脚本轮询 /auth/login.action	tax-jee-app 与 tax-jee-mysql 均为 healthy
版本和禁止项容易回退	增加 verify-baseline.ps1，扫描构建版本、WAR 依赖与基线配置	输出 Java EE 8 baseline check PASSED

3. 最终部署工件

工件	仓库路径
Docker 构建	Dockerfile
服务编排	compose.yaml
部署说明	DEPLOY.md
数据库结构	database/schema.sql
演示数据	database/seed-demo.sql
环境变量模板	.env.docker.example
基线验证	scripts/verify-baseline.ps1

4. 部署决策说明

最终采用 Docker Compose 部署，是因为课程要求固定 Tomcat 9 与 MySQL 8，而 Docker 能把这组版本与数据库初始化一并封装，避免不同机器上的 JDK、Servlet 容器和数据库差异。该方式适合教学演示、课程验收和需要快速重建的环境；不适合没有 Docker 运行时的受限机房，也不适合要求高可用和独立运维监控的生产系统。项目坚持 Java EE 8 与 Tomcat 9，是因为全部代码、API 和部署产物都依赖 javax.servlet 4.0，升级到更高容器会改变命名空间并超出课程边界。



图 7-1 Docker 部署后的登录页

5. AI 加固记录

建议	兼容性判断	结果
固定 Temurin 11 镜像	兼容编译目标 Java 11	采纳并实跑
使用 MySQL 8.0.46 镜像	满足 MySQL 8 且锁定补丁版本	采纳并初始化 13 表
把 Tomcat 升级到最新版	会越过课程 Tomcat 9 边界	拒绝

实验总结

部署加固不只是增加启动脚本，而是把版本、依赖、数据库凭据、健康检查、初始化数据和故障说明写成可验证的契约。当前项目的容器状态、页面响应和 WAR 依赖扫描都能重复执行。

附上代码文件

```
Dockerfile
compose.yaml
.env.docker.example
DEPLOY.md
scripts/verify-baseline.ps1
docs/version-baseline.md
```

实验八：GitHub 最终项目提交与课程总结

技术基线：Java EE 8（javax.servlet.*） | Tomcat 9 | JDK 11 | Maven 3.8+ | MySQL 8

姓名	吴学孟	学号	24110741085
课程名称	Java EE	班级	24 软工 2 班
日期	2026 年 11 月 10 日	得分	
题目或任务	**税务业务管理系统** 任务 1：核对 README、DEPLOY.md、源码和数据库脚本 任务 2：确认 Java EE 8 技术边界 任务 3：整理功能演示与业务规则证据 任务 4：完成 AI 使用总结 任务 5：提交仓库 URL 与最终 commit		

解答及实验步骤

1. 最终仓库检查

检查项	结果	证据
README 技术栈与功能	已更新	根目录 README.md
DEPLOY.md 从零部署	已提供	DEPLOY.md
源码与数据库脚本	完整	src/、database/
Java EE 8 命名空间	通过	Maven 依赖树仅 javax.servlet
Tomcat 9 / MySQL 8	通过	Tomcat 9.0.123 / MySQL 8.0.46
禁止 Jakarta 与 Tomcat 10+	通过	git grep 无 Jakarta 包引用
有意义 commit	18 次	git rev-list --count HEAD
配对部署证据	待搭档确认	最终提交前补齐签名或截图

2. 功能演示与业务规则



图 8-1 工作台概览

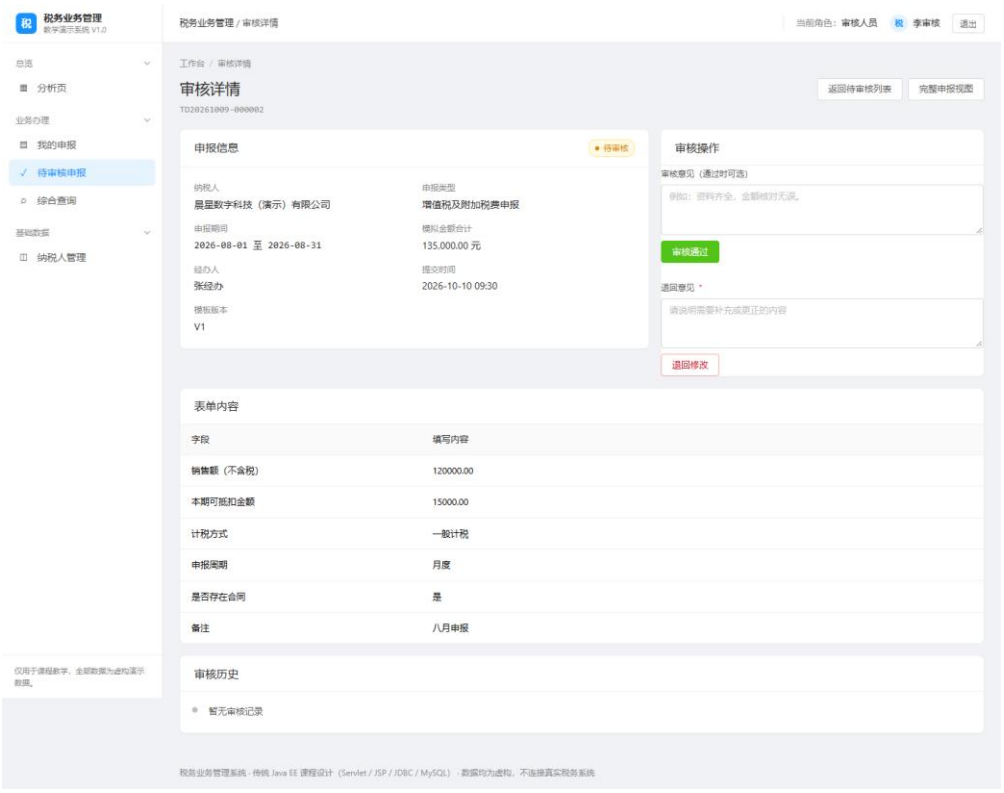


图 8-2 申报详情与审核记录

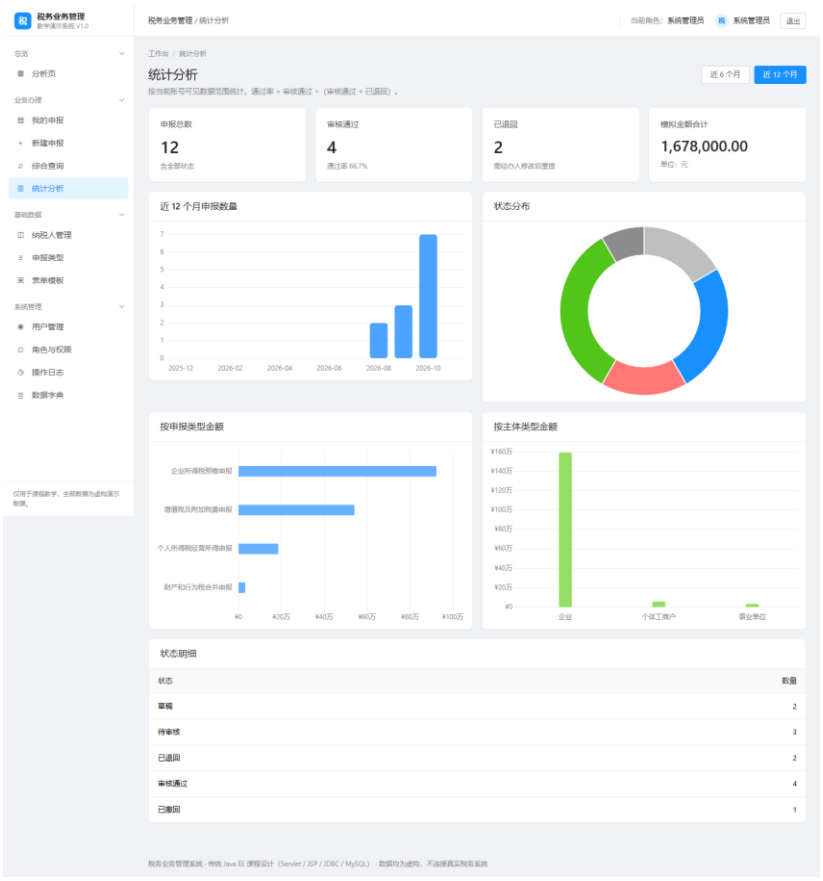


图 8-3 业务统计分析

3. AI 使用总结

在实验四至七中，我最信任 AI 的环节是依赖清单核对、重复性文档整理、命令示例和故障线索归纳，因为这些结果可以立刻通过编译、依赖树、容器版本和 HTTP 响应验证。最不信任的是涉及业务边界和运行版本的判断，例如容器 JDK、Servlet 命名空间、权限规则和数据库状态；这些内容一旦看似正确却实际偏离基线，补救成本高于手工核对。

我的固定流程是：先给出明确的 Java EE 8、Tomcat 9、JDK 11 约束；要求 AI 输出可检查的改动点；人工逐项审查是否引入其他技术栈；执行 mvn test、构建 WAR、依赖扫描和 Docker 健康检查；最后把命令、结果和失败分支写回文档。生成文本本身不作为完成证据，只有运行验证之后的代码和输出才进入结论。

本项目中，AI 曾推动过更高版本 JDK 和双技术栈方向的建议，我通过课程要求和技术边界识别冲突，将课程仓库收窄为传统 Java EE 8 项目，并将 Docker 运行环境固定为 Temurin 11。如果下一届学生使用 AI 学 Java EE，最重要的一条警告是：先固定课程要求的技术边界，再让 AI 写代码；不要先接受一套更现代的默认答案，再试图事后证明它属于 Java EE 8。

4. 配对部署反思

本次工作聚焦本人项目，未用不存在或未经确认的搭档部署结果填充报告。这反而暴露了一个关键问题：开发者熟悉自己的目录、端口和数据库密码，容易把隐含前提留在脑内。对陌生部署者而言，JDK 小版本、Tomcat 主版本、WAR 上下文、容器健康状态和 MySQL 初始化顺序都可能成为阻塞点。完善 DEPLOY.md 后，部署文档不再只是命令列表，而是包含版本、预期输出、失败分支和验证 URL 的执行契约。

5. 最终提交信息

项目	内容
GitHub URL	https://github.com/Caspareilo/tax-business-mangement
最终 commit	a197007
分支	main
提交总数	18
自动化测试	30 个全部通过
Docker 状态	应用与 MySQL 均 healthy

实验总结

Java EE 8 固定栈让 AI 的风险变得可识别：偏离 javax.servlet、Tomcat 9 或 JDK 11 的建议可以依据配置和运行证据直接拒绝。AI 对效率的帮助最大，风险最大的部分则是把“常见的现代 Java 写法”误当作课程允许写法。

附上代码文件

```
README.md
DEPLOY.md
pom.xml
Dockerfile
compose.yaml
scripts/verify-baseline.ps1
docs/version-baseline.md
```